

Capstone Project Report: Social Network Explorer (SNE)

Course: Basics of Data Structures (ETCCBD201)

Student Name: Diya Kumal

Program: B.Sc (H) CS

Roll No: 2501720001

1. Project Design & Modular Architecture

To meet industry standards for clean code and modularity, the SNE system is divided into distinct functional modules:

`profiles.py`: Manages user data using hashing for efficient $O(1)$ average-case retrieval.

`network_graph.py`: Implements the friendship network using an adjacency list.

`algorithms.py`: Contains the core logic for BFS (Shortest Path) and DFS (Depth-Limited Search).

`sorting.py`: A dedicated module for ranking friend suggestions using custom sorting algorithms.

`main.py`: The central driver that handles the CLI menu and the mandatory Demo Mode.

2. Data Structure Implementation

2.1 User Profile Management (Hashing)

User profiles are stored in a Hash Table to facilitate rapid access. Each profile entry consists of a unique `user_id`, name, age, and a list of interests.

Add User: Inserts a new profile after validating that the `user_id` is unique.

Get/Update: Uses the hash key for near-instantaneous retrieval and modification of user details.

2.2 Friendship Network (Graphs)

The system models social connections as a graph using an Adjacency List. This approach is memory-efficient for sparse social networks where each user is connected to only a fraction of the total population.

Connections: Supports bidirectional friendships where `graph[u1]` includes `u2` and vice versa.

3. Algorithmic Analysis (Relationship Discovery)

3.1 BFS: Shortest Path

Breadth-First Search (BFS) is used to find the "Degrees of Separation" between two users.

Logic: It utilizes a Queue (Unit 2) to explore neighbors level-by-level, ensuring the first time the target user is reached, it is via the shortest possible path.

Complexity: $O(V + E)$, where V is users and E is connections.

3.2 DFS: Friends-of-Friends

Depth-First Search (DFS) is implemented to discover all connections within a specific depth k (e.g., depth 2 or 3).

Logic: Uses Recursion and a "visited" set to prevent infinite loops and redundant processing.

4. Recommendation System & Sorting (Unit 3)

The recommendation engine identifies potential friends by comparing shared interests.

Conceptual Approach: Hashing can be used to create an "inverted index" where interests map directly to lists of user IDs for rapid grouping.

Scoring: The system calculates a "Common Interest Score" between a target user and all non-connected users.

Ranking: Candidates are sorted in descending order using Insertion Sort or Merge Sort to provide the top 5 suggestions.

5. Complexity Summary Table

Operation	Data Structure	Average Complexity
Add/Retrieve Profile	Hash Table	$O(1)$
Shortest Path	BFS (Queue)	$O(V + E)$
Depth Search	DFS (Recursion)	$O(V + E)$
Ranking Suggestions	Insertion Sort	$O(n^2)$

6. Demo Mode Output Summary

The program includes a mandatory demo mode that executes the following automatically:

1. Setup: Inserts 6–8 users and establishes 8–12 connections.
2. Discovery: Performs 2 BFS queries and a DFS search at depth 2 and 3.
3. Recommendations: Displays the top-5 suggested friends for a sample user based on interest scores.

References

ETCCBD201 Course Syllabus: Unit 1 to Unit 4.

Industry Standard Modular Design Patterns for Python.